module2_playback_engine

Module 2 — Playback Engine Architecture

Virtual Harmonium Lab · ESP32 Digital Musical Workstation

Engineering Reference Document

Source: Polyphone 2.x Source Code (© 2013–2024 Davy Triponney, GPL v3)
Date compiled: 2026-07-27
Status: Authoritative · Do not modify without re-verifying source

---

> Module 1 answered: **"What information defines an instrument?"**
> Module 2 answers: **"How is that information transformed into real-time audio?"**

Table of Contents

1. Complete Playback Pipeline
2. Thread Architecture
3. Voice Creation and Lifecycle
4. Region Selection
5. Sample Selection
6. Pitch Engine
7. Sample Playback and Resampler
8. Loop Engine
9. Envelope Engine
10. LFO Engine
11. Filter Engine
12. Modulator Runtime
13. DSP Processing Order
14. Mixer
15. Voice Management and Polyphony
16. Runtime Object Reference
17. Source Code Extraction Reference

18. Timing Analysis
19. Runtime Memory
20. Embedded Notes Summary

---

1. Complete Playback Pipeline

Full Pipeline — Note On to Audio Output

```
┌──────────────────────────────────────────────────────────────────────┐
│ THREAD 1: MIDI / GUI THREAD                                          │
│                                                                      │
│  MIDI Note-On (key, vel, channel)                                    │
│         │                                                            │
│         ▼                                                            │
│  Synth::play(EltID, channel, key, velocity)                          │
│         │                                                            │
│         ▼                                                            │
│  Preset Lookup                                                       │
│  Synth::playPrst()                                                   │
│    For each preset Division where keyRange matches AND               │
│    velRange matches:                                                 │
│         │                                                            │
│         ▼                                                            │
│  Instrument Lookup                                                   │
│  Synth::playInst()                                                   │
│    For each instrument Division where keyRange matches AND           │
│    velRange matches:                                                 │
│         │                                                            │
│         ▼                                                            │
│  Sample Retrieval                                                    │
│  smpl→_sound.loadSample()     [lazy load from disk]                  │
│         │                                                            │
│         ▼                                                            │
│  Voice Allocation                                                    │
│  VoiceList::getVoiceToPrepare()  [ring buffer, no malloc]            │
│         │                                                            │
│         ▼                                                            │
│  Voice Initialization                                                │
│  Voice::initialize()                                                 │
│    ├─ VoiceParam::initialize()                                       │
│    │    ├─ readSmpl()        (root key, loop, length, fine tune)     │
│    │    ├─ readDivisionAttributes(instGlobal, instDiv)               │
│    │    ├─ readDivisionAttributes(prstGlobal, prstDiv)               │
```

```
|    |    ├─ initialize ModulatorGroups (inst + prst)                |
|    |    ├─ readDivisionModulators (inst + prst)                    |
|    |    ├─ initializeModulatorInputs()                             |
|    |    └─ computeModulations()   (initial parameter values)       |
|    ├─ LFO initialization  (_modLFO, _vibLFO)                       |
|    └─ Envelope initialization (_volEnvelope, _modEnvelope)         |
|         |                                                          |
|         ▼                                                          |
|  Synth::configureVoice()  (master gain, pan for sample mode)       |
|         |                                                          |
|         ▼                                                          |
|  VoiceList::addPreparedVoices()  [atomic publish to audio]         |
└────────────────────────────────────────────────────────────────────┘

┌────────────────────────────────────────────────────────────────────┐
| THREAD 2: SOUND ENGINE WORKER THREAD(S)                            |
|                                                                    |
|  SoundEngine::generateData(bufferLen)       [per buffer]           |
|                                                                    |
|  For each active Voice:                                            |
|         |                                                          |
|         ▼                                                          |
|  Voice::generateData(voiceBuffer, bufferLen)                       |
|                                                                    |
|  For each CHUNK (256 samples):                                     |
|         |                                                          |
|         ├─ 1. Vol Envelope chunk value  (_dataVolArray[chunk])     |
|         ├─ 2. Mod Envelope chunk value  (_dataModArray[chunk])     |
|         ├─ 3. Mod LFO chunk value       (_modLfoArray[chunk])      |
|         ├─ 4. Vib LFO chunk value       (_vibLfoArray[chunk])      |
|         ├─ 5. Compute pitch delta (cents) from:                    |
|         |       coarseTune + fineTune + scaleTuning +              |
|         |       modEnv*modLFO*vibLFO modulation                    |
|         ├─ 6. Convert cents → fixedGap (fixed-point ratio)         |
|         ├─ 7. Sinc resampling  (fast path or slow/loop path)       |
|         ├─ 8. Compute volumeCoeff (envelope * LFO volume)          |
|         ├─ 9. Compute filter coefficients (per chunk)              |
|         └─ 10. Apply biquad filter + volume scaling (per sample)   |
|                                                                    |
|  SoundEngine applies pan + reverb/chorus routing:                  |
|    coefL = cos(pan), coefR = sin(pan)                              |
|    Accumulate into 8 output buses:                                 |
|      L, R, RevL, RevR, ChoL, ChoR, ChoRevL, ChoRevR                |
└────────────────────────────────────────────────────────────────────┘

┌────────────────────────────────────────────────────────────────────┐
| THREAD 3: AUDIO CALLBACK THREAD                                    |
```

```
│                                                                    │
│  Synth::readData(dataL, dataR, maxlen)     [per buffer]            │
│    ├─ Stop current SoundEngine processing (_endComputation)        │
│    ├─ Collect results: setData/addData from all SoundEngines       │
│    ├─ prepareComputation() – voice management + exclusive class    │
│    ├─ Restart SoundEngines: prepareData(maxlen)                    │
│    ├─ applyChoRev() – reverb + chorus effect processing            │
│    ├─ LiveEQ::filterData() – optional 10-band EQ                   │
│    ├─ CalibrationSinus::addData() – optional pitch sinus           │
│    ├─ FastMaths::clamp(dataL, dataR) – hard clipping to [-1,1]     │
│    └─ Recorder::process() – optional file recording                │
└────────────────────────────────────────────────────────────────────┘


                    OUTPUT BUFFER (dataL, dataR)
```

---

2. Thread Architecture

Polyphone uses three threads for audio generation:

Thread 1: MIDI / GUI Thread

Handles:

- MIDI input events (`processKey` , `processControllerChanged` , `processBendChanged` , etc.)
- Voice creation calls (`Synth::play` , `Synth::playPrst` , `Synth::playInst` , `Synth::playSmpl`)
- Writing new voices into the `VoiceList` ring buffer

Thread 2: SoundEngine Worker Thread(s)

There are `QThread::idealThreadCount() - 1` worker threads, at minimum 1:

```
// synth.cpp line 109
_soundEngineCount = qMax(QThread::idealThreadCount() - 1, 1);
```

Each `SoundEngine` runs on its own `QThread` at `HighPriority` . Each thread loops indefinitely, calling `generateData()` when released by the mutex. Threads pull voices from the shared `VoiceList` using an atomic index (`_currentIndex.fetchAndAddOrdered(1)`) so that each voice is processed by exactly one thread with no duplication.

Thread 3: Audio Callback Thread

Driven by the audio server (e.g., JACK, ALSA, PortAudio). Calls `Synth::readData()` exactly once per audio buffer. This function:

1. Stops the ongoing SoundEngine computation
2. Collects the just-rendered data from all engines (copying + accumulating)
3. Runs voice management (closes finished voices, handles exclusive classes, enforces polyphony limit)
4. Restarts the SoundEngines for the next buffer
5. Applies post-processing (reverb, chorus, EQ, clipping, recording)

Synchronization Design

- `VoiceList` uses atomic integers for lock-free voice handoff between threads
- A `QMutex _mutexSynchro` in each `SoundEngine` is used to gate when computation begins
- The `_mutexSoundfonts` protects soundfont data during note-on (Thread 1 only)
- No mutex is used during audio generation — the ring buffer design ensures safe concurrent access

Source: `sources/sound_engine/synth.cpp` lines 97–128, `sources/sound_engine/soundengine.cpp` lines 79–96

Embedded Notes — Thread Architecture

> **ESP32 Relevance:** The multi-thread design is irrelevant to ESP32.
> On ESP32, a single-threaded ISR-driven or DMA-driven audio callback is the correct model.
> The VHL playback engine should use a single audio fill function called once per DMA buffer.
> No worker threads, no mutexes, no ring buffers needed.

---

3. Voice Creation and Lifecycle

What a Voice Is

A `Voice` is the fundamental runtime unit. One voice = one playing note from one sample zone. It is completely self-contained: it holds all DSP state needed to generate its contribution to the output buffer independently of all other voices.

Voice Allocation — Ring Buffer

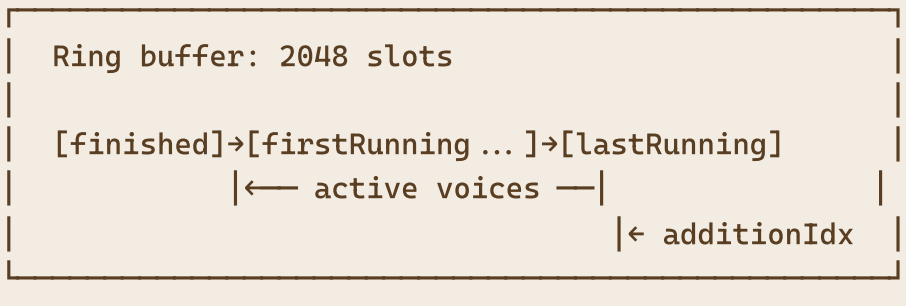
All voices are **pre-allocated at startup**. There is no `new Voice()` call during playback.

```
// voicelist.cpp lines 55-62
for (int i = 0; i < MAX_NUMBER_OF_VOICES; i++) {
    _voiceParameters[i] = new VoiceParam();
    _voices[i] = new Voice(_voiceParameters[i]);
}
```

`MAX_NUMBER_OF_VOICES = 2048`. All 2048 `Voice` objects and their associated `VoiceParam` objects exist for the entire lifetime of the application.

The `VoiceList` manages these voices as a **circular ring buffer** with multiple indexed regions:

- `_firstRunningIndex` — first active voice (inclusive)
- `_lastRunningIndex` — last active voice (exclusive)
- `_additionIndex` — next slot to be filled by MIDI thread
- `_preparationIndex` — temporarily claimed by MIDI thread while building a note



When a note-on arrives, `getVoiceToPrepare()` claims the next slot (incrementing `_preparationIndex`). When all zones for one note-on event have been initialized, `addPreparedVoices()` commits them atomically by moving `_additionIndex`.

Voice Initialization

```
// Called from Synth::playSmpl(), synth.cpp line 291-293
voice->initialize(prst, prstDiv, inst, instDiv, smpl,
                  channel, key, velocity, type,
                  _internalConfiguration.sampleRate, currentToken);
```

`Voice::initialize()` resets all DSP state and calls:

1. `_voiceParam->initialize( ... )` — loads all 60 parameters, runs modulators

2. `smpl->_sound.getData( ... )` — gets pointer to PCM data (or copies it at sample level)
3. Resets `_currentSmplPos`, `_release`, `_isFinished`, filter states, volume coefficient
4. Initializes the LFO objects (`_modLFO.initialize(chunkRate)`, `_vibLFO.initialize(chunkRate)`)
5. Initializes the envelope objects (`_volEnvelope.initialize(chunkRate, false)`, `_modEnvelope.initialize(chunkRate, true)`)
6. Zeroes the sinc resampling buffers

Voice Lifetime

Event	What Happens
Note On	`Voice::initialize()` called, voice enters active ring
Note Off	`Voice::release()` called → `_release = true`, envelope enters Release phase
Exclusive class fires	`Voice::release(quick=true)` → `_volEnvelope.quickRelease()` → 40ms release
Sample ends (no loop)	`Voice::getData()` detects end → `Voice::finish()` called
Envelope reaches Off	`EnveloppeVol::getEnvelope()` returns `true` → `_isFinished = true`
Polyphony exceeded	`VoiceList::prepareComputation()` forces `voice->finish()` on oldest released voices
Audio callback	`VoiceList::prepareComputation()` removes finished voices from the ring

Voice Type

`VoiceParam::_type` encodes the context of the voice:

- `0` — normal instrument/preset voice
- `1` — direct sample playback (main channel)
- `2` — stereo part of direct sample playback

Only type `0` voices participate in modulators. Type 1/2 voices copy sample data and use simplified parameters.

Source: `sources/sound_engine/voicelist.cpp`, `sources/sound_engine/voice.cpp` lines 78–126

Embedded Notes — Voice Lifecycle

> **ESP32 Relevance:** The pre-allocation strategy is exactly right for embedded systems.
> VHL should pre-allocate all voice objects at boot.
> The ring buffer design is elegant but overkill for ESP32; a simple flat array of N voices with a "used" flag is sufficient.
> The key insight: `new` / `delete` must never be called during audio processing.

---

4. Region Selection

Where It Happens

Region selection is performed on the MIDI/GUI thread during `Synth::play()`, not during audio generation. This is critical: by the time audio is generated, the correct zones have already been identified.

Two-Stage Selection

Stage 1: Preset Zone Selection (`Synth::playPrst`)

```
// synth.cpp lines 196-229
// 1. Read global key/vel ranges from the preset's global division
// 2. For each preset zone (Division):
//    a. Skip hidden or muted zones
//    b. currentKeyRange = zone's keyRange if set, else global keyRange
//    c. currentVelRange = zone's velRange if set, else global velRange
//    d. If key ≥ byLo AND key ≤ byHi AND vel ≥ byLo AND vel ≤
byHi:
//       → get the linked instrument and call playInst()
```

Stage 2: Instrument Zone Selection (`Synth::playInst`)

```
// synth.cpp lines 233-274
// 1. Read global key/vel ranges from the instrument's global division
// 2. For each instrument zone (Division):
//    a. Skip hidden or muted zones
//    b. currentKeyRange = zone's keyRange if set, else global keyRange
//    c. currentVelRange = zone's velRange if set, else global velRange
//    d. If key ≥ byLo AND key ≤ byHi AND vel ≥ byLo AND vel ≤
byHi:
//       → get the linked sample and call playSmpl()
```

Multiple Active Zones — All Fire

When multiple zones have overlapping ranges that contain the pressed key and velocity, **all matching zones fire simultaneously**. Each produces a separate Voice. This is the correct SF2 behavior and enables:

- Layering (multiple samples playing at the same pitch for richness)
- Velocity crossfades (both zones active, volume determined by velocity-based attenuation)
- Round-robin (not in SF2 standard, but possible via exclusive classes)

No Zone Priority

There is no "nearest match" or "best match" logic. Every zone whose range contains `{key, vel}` produces a voice. This is different from some simpler samplers that pick only one zone.

Zone Skip Conditions

```
Division * prstDiv = divisions[i];
if (prstDiv->isHidden() || prstDiv->isMute())
    continue;
```

`_mute` is an editor flag, `_hidden` is for the global division (it has no sampleID). In practice, both prevent zone playback.

Embedded Notes — Region Selection

> **ESP32 Relevance:** The two-stage preset→instrument traversal can be simplified.
> For VHL, we will pre-bake the full chain at conversion time.
> Zone lookup becomes a single pass through a flat array of zone descriptors.
> Key/vel range check = two `if` statements. Extremely cheap.
> The "all matching zones fire" behavior must be preserved — it is essential for layered sounds.

---

5. Sample Selection

How a Sample is Retrieved

After a zone is matched, the sample index stored in `champ_sampleID` (generator 53) is used:

```
// synth.cpp line 268-271
Smpl * smpl = soundfont->getSample(instDiv-
>getGen(champ_sampleID).wValue);
if (smpl != nullptr)
    this->playSmpl(smpl, channel, key, velocity, false, inst, instDiv,
prst, prstDiv);
```

The sample is retrieved by index from the `Soundfont::_smpl` list. No search, no distance matching — it is a direct lookup.

Lazy Sample Loading

Before the voice is initialized, the sample data is lazily loaded:

```
// synth.cpp line 281
smpl->_sound.loadSample();
```

`Sound::loadSample()` reads the PCM data from disk only if it has not been loaded yet. Once loaded, the data stays in memory permanently until the soundfont is unloaded.

Sample Data Access

At voice initialization, the voice gets a pointer (or copy) to the PCM data:

```
// voice.cpp line 93
smpl->_sound.getData(_dataSmplLength, _dataSmpl16, _dataSmpl24,
                     false, _voiceParam->getType() != 0 /* copy at
sample level */);
```

- For instrument/preset voices (type 0): a **pointer** to the shared PCM data is stored. No copy.
- For direct sample playback voices (type 1/2): a **copy** of the data is made. This allows the editor to modify loop points live.

There Is No "Nearest Sample" Logic

Unlike some hardware synths, Polyphone does not select the nearest sample when a zone is not matched. Zone selection is binary: either the key/vel is inside the range (fire) or it is not (skip). The "nearest sample" concept is a feature of some sample-based instruments (like Kontakt's "sample mapping"), not SF2.

Multiple Samples Simultaneously

Yes — every matching zone from every matching preset/instrument produces a separate voice. A single key press can generate dozens of voices. All play simultaneously. Their outputs are mixed additively.

Stereo Samples

When a sample has `sfSampleType` of `leftSample` or `rightSample`, the linked (partner) sample is automatically triggered too:

```
// synth.cpp lines 159-163
if (smpl->_sfSampleType != monoSample && smpl->_sfSampleType !=
RomMonoSample) {
    Smpl * otherSmpl = soundfont->getSample(smpl->_wSampleLink);
    if (otherSmpl != nullptr)
        playSmpl(otherSmpl, channel, key, velocity, true);
}
```

The stereo partner gets `other = true`, which sets `_type = 2` and assigns pan = -500 (left) or +500 (right).

Embedded Notes — Sample Selection

> **ESP32 Relevance:** Direct index lookup is the correct approach for ESP32 — no search needed.
> Stereo playback: on a memory-constrained ESP32, stereo doubles the RAM and voice count needed.
> For VHL, a mono-only instrument bank is a practical starting point.
> Lazy loading from disk: on ESP32, samples will be loaded from PSRAM or flash at startup and stay there permanently.

---

6. Pitch Engine

Overview

Pitch is computed inside `Voice::generateData()` once per chunk (256 samples), not once per sample. The result is a `fixedGap` — an unsigned 11-bit fixed-point integer representing how many source samples to advance per output sample.

The Pitch Formula

The core pitch calculation (voice.cpp lines 201–207):

```
float deltaPitchFixed =
    -1731.234f /* = -1200 / ln(2) */ *
        qLn(static_cast<double>(_audioSmplRate) * 4400.f / (_smplRate *
s_referencePitch))
    + (playedNote - v_rootkey) * v_scaleTune
    + (temperamentFineTune + static_cast<float>(v_fineTune))
    + 100.0f * v_coarseTune;
```

Breaking it down:

Term	Meaning
`-1731.234f * ln(audioRate * 4400 / (smplRate * refPitch))`	Base pitch shift in cents due to sample rate mismatch and reference pitch (4400 = A4=440Hz × 10)
`(playedNote - v_rootkey) * v_scaleTune`	Pitch shift for interval from root key, scaled by tuning scale (100 = equal temperament, 0 = no pitch change)
`temperamentFineTune + v_fineTune`	Temperament correction in cents + generator fine tune in cents
`100.0f * v_coarseTune`	Coarse tune in semitones × 100 = cents

Note: The sample's `_sampleFineTune` (champ_chPitchCorrection) is added separately inside `VoiceParam::getInteger(champ_fineTune)`:

```
return (type == champ_fineTune ? _sampleFineTune : 0) +
_parameters[type].getIntValue();
```

So the full fine tuning is: `sample fine tune + zone/preset fine tune generator + modulations`.

Per-Chunk Pitch with Modulation

Every chunk, the pitch delta is updated to include LFO and envelope modulations:

```
// voice.cpp line 226
float currentDeltaPitch = deltaPitchFixed
    + v_modEnvToPitch * _dataModArray[chunk]       // mod envelope →
pitch
    + v_modLfoToPitch * _modLfoArray[chunk]        // mod LFO → pitch
    + v_vibLfoToPitch * _vibLfoArray[chunk];       // vib LFO → pitch
```

`_dataModArray[chunk]` is in range [0, 1] (volume envelope output) or [-1, 1] (modulation context).
`v_modEnvToPitch` is in cents (integer).

Fixed-Point Resampling Ratio

The pitch delta in cents is converted to a fixed-point integer:

```
// voice.cpp line 227-228
quint32 fixedGap = static_cast<quint32>(
    FastMaths::fastPow2(currentDeltaPitch * 0.000833333f /* 1/1200 */ +
11.0f /* shift left by 2048 */) + 0.5f);
```

The formula: `fixedGap = round(2^(deltaCents/1200 + 11))`

- `2^11 = 2048` is the fixed-point scale factor
- When `deltaCents = 0` (root key, same sample rate, no transpose): `fixedGap = 2048`
- When `deltaCents = 1200` (one octave up, need to advance 2x faster): `fixedGap = 4096`
- When `deltaCents = -1200` (one octave down): `fixedGap = 1024`

This gives 11 bits of integer part and the remaining bits encode the sub-sample fractional position.

Sinc Table Selection

Different sinc window shapes (Kaiser-Bessel windows with beta 4.3 to 7.8) are selected based on the sample's root key and the pitch shift:

```
// voice.cpp lines 499-508
int Voice::whichSincTable(unsigned char sampleRootKey, double shift)
{
    const double changeByRootKey = 0.03;
    const double changeByShift = 0.3;
    int index = static_cast<int>(3.0 + changeByRootKey *
((double)sampleRootKey - 64.)
                                     + changeByShift * shift + 0.5);
    return qBound(0, index, 7); // Clamp to [0, 7]
}
```

Lower root key or larger upward shift → sharper anti-aliasing filter needed → higher beta window. This prevents aliasing artifacts when upsampling high-frequency samples.

Source: `sources/sound_engine/voice.cpp` lines 196–231

Embedded Notes — Pitch Engine

> **ESP32 Relevance:** The formula is straightforward but requires `pow2()` and `ln()` calls.
> For ESP32, pre-compute `fixedGap` from a lookup table indexed by (playedNote - rootKey) or use fixed-point exponentiation.
> The sample-rate compensation term is static per voice — compute once at note-on, not per chunk.
> Modulation additions (LFO, mod env) are per-chunk — keep these as integer additions in cents, convert to ratio only when needed.

---

7. Sample Playback and Resampler

Playback Pointer

The voice maintains:

- `_currentSmplPos` (quint32): current integer position in the source sample array
- `_lastDistanceFraction` (quint32): fractional sub-sample position, fixed-point with 11 bits representing the fraction (values 0–2047)
- `_moreAvailable` (qint32): how many full source samples are available from the current position before a boundary (end-of-sample or loop point)

Sinc Interpolation

Polyphone uses a **polyphase sinc interpolation** resampler. This is a high-quality fractional-delay filter.

Configuration:

- `RESAMPLING_ORDER = 8` : filter length (8 taps used per output sample)
- `RESAMPLING_SUBDIVISION = 256` : number of fractional phases stored in the table
- 8 different sinc window tables (`s_sinc_tables[8][256][8]`), pre-computed at startup

Each output sample is computed as:

```
output = Σ (sinc_coeff[k] × input[currentPos + k])  for k = 0..7
```

Where the sinc coefficients are selected by the fractional phase of the playback cursor: `(_lastDistanceFraction >> 3) & 0xFF` gives the index into the 256-phase table.

The Resampling Inner Loop (Fast Path):

```
// voice.cpp lines 239–262
while (true) {
    _lastDistanceFraction += fixedGap;        // Advance fractional
position
    nextPosition = (_lastDistanceFraction >> 11);  // Integer advance

    // Fast path: enough source data without boundary check
    (*pData++) = FastMaths::multiply8(
        _sincWindow[(_lastDistanceFraction >> 3) & 0xFF],
        &srcData16[nextPosition],
        &srcData24[nextPosition]);

    if (pData ≥ pDataEnd) { ... goto resample_end; }
}
```

`FastMaths::multiply8()` computes the 8-tap dot product using SIMD or unrolled scalar multiplication, combining 16-bit and 24-bit (extra byte) data into one float output.

Slow Path: Used at boundaries (end of sample or near loop points):

- Calls `Voice::getData()` or `Voice::getDataWithLoop()` to manage the position
- Copies boundary data into a scratch buffer `_srcData16[8]` to allow the 8-tap filter to read beyond boundaries safely

Integer → Float Conversion

The 24-bit sample value conversion:

```
// voice.cpp line 46
s_floatConversionCoef24 = 1.0f / (0.5f + static_cast<float>(0x7FFFFF));
```

This normalizes the combined 16+8 bit value to the range approximately [−1, +1].

Source: `sources/sound_engine/voice.cpp` lines 237–325

Embedded Notes — Resampler

> **ESP32 Relevance:** 8-tap sinc interpolation is heavy for ESP32 (8 multiplications + 8 accumulations per output sample).
> For VHL, linear interpolation (2-tap) is the practical first choice: `output = sample[n] + frac * (sample[n+1] - sample[n])`.
> Sinc can be added as a quality option later.
> The sub-sample fractional cursor (11-bit fixed-point fraction) concept should be preserved exactly — this is the correct way to implement variable-ratio resampling without floating-point loop counters.

---

8. Loop Engine

Loop Modes

`champ_sampleModes` (generator 54) controls loop behavior:

Value	Mode	Behavior
`0`	No loop	Play once from start to end, then voice finishes
`1`	Continuous loop	Loop forever between `loopStart` and `loopEnd` (both while key is held and during release)
`2`	No loop until note-off	Plays silently until Note Off, then plays from start (release trigger mode)
`3`	Loop while key held, no loop in release	Loops while key held; when Note Off arrives, exits loop and plays to end

Note: There is no "bidirectional (ping-pong) loop" in SF2/Polyphone. Only forward looping is implemented.

Loop Selection at Chunk Boundary

At the start of each chunk, the correct data extraction function is selected:

```
// voice.cpp line 235
void(Voice::*slowExtractFunction)
(quint32,quint32,quint32,qint16*&,quint8*&) =
    (_loopMode == 1 || (_loopMode == 3 && !_release))
        ? &Voice::getDataWithLoop
        : &Voice::getData;
```

- Mode 1: always use `getDataWithLoop`
- Mode 3 and key held: use `getDataWithLoop`; mode 3 and released: use `getData` (play to end)
- Modes 0, 2: always use `getData`

Loop Boundary Validation

Before entering the loop:

```
// voice.cpp lines 210-213
quint32 loopStart = _voiceParam->getPosition(champ_dwStartLoop);
quint32 loopEnd   = _voiceParam->getPosition(champ_dwEndLoop);
if (loopStart ≥ loopEnd && _loopMode ≠ 2)
    _loopMode = 0;  // Invalid loop → force no-loop
```

If `loopStart ≥ loopEnd`, the loop is invalid and automatically disabled.

Loop Wrap-Around Algorithm

```
// voice.cpp lines 388-429
void Voice::getDataWithLoop(quint32 goOn, quint32 loopStart, quint32
loopEnd,
                            qint16 *&srcData16, quint8 *&srcData24)
{
    _currentSmplPos += goOn;

    // Wrap back to start of loop if past the end
    while (_currentSmplPos ≥ loopEnd)
        _currentSmplPos += loopStart - loopEnd;  // Equivalent to: -=
(loopEnd - loopStart)

    // If enough samples are available without crossing loopEnd:
    if ((_moreAvailable = loopEnd - _currentSmplPos - RESAMPLING_ORDER)
≥ 0) {
        srcData16 = &_dataSmpl16[_currentSmplPos];
        //  ...  (fast pointer path)
    }
    else {
        // Cross-boundary case: fill scratch buffer from end + start of
loop
        quint32 remainingSamples = loopEnd - _currentSmplPos;
        memcpy(_srcData16, &_dataSmpl16[_currentSmplPos],
remainingSamples * sizeof(qint16));
        memcpy(&_srcData16[remainingSamples], &_dataSmpl16[loopStart],
               (RESAMPLING_ORDER - remainingSamples) * sizeof(qint16));
```

```
        // ... same for 24-bit extension
    }
}
```

When a loop is shorter than `RESAMPLING_ORDER` (8 samples), the scratch buffer is filled by cycling through the loop multiple times until 8 samples are collected.

Loop Point Offsets

The actual runtime loop points include generator offsets:

```
// voiceparam.cpp lines 357–381
// champ_dwStartLoop:
offset = _parameters[champ_startloopAddrsOffset].getIntValue()
       + 32768 *
_parameters[champ_startloopAddrsCoarseOffset].getIntValue();
result = _sampleLoopStart + offset;

// champ_dwEndLoop:
offset = _parameters[champ_endloopAddrsOffset].getIntValue()
       + 32768 *
_parameters[champ_endloopAddrsCoarseOffset].getIntValue();
result = _sampleLoopEnd + offset;
```

Source: `sources/sound_engine/voice.cpp` lines 385–429

Embedded Notes — Loop Engine

> **ESP32 Relevance:** The loop wrap-around algorithm is exactly what is needed for ESP32.
> The `while (_currentSmplPos ≥ loopEnd) _currentSmplPos += loopStart - loopEnd;` idiom is safe and correct even for short loops.
> Mode 2 (release trigger) is rare — can be omitted from VHL's first version.
> Mode 3 (loop during sustain only) is very important for harmonium reeds — implement this.
> Loop point offsets (startloopAddrsOffset etc.) can be pre-baked at conversion time to avoid runtime addition.

---

9. Envelope Engine

Overview

Polyphone uses one `EnveloppeVol` class for both the **volume envelope** and the **modulation envelope**. The behavior is identical in structure but differs in sustain unit and attack/decay shapes:

Property	Volume Envelope	Modulation Envelope
Sustain unit	dB attenuation (0 = full, 144 = silence)	Percentage (0 = 0%, 100 = 100%)
Attack shape	Linear amplitude (convex in dB)	Convex (asymptotic exponential approach)
Decay shape	Exponential	Linear
Release shape	Exponential	Linear
Key scaling applies?	Yes (hold + decay time scaled by key number)	Yes

State Machine

The `EnveloppeVol` has 8 states:

```
phase1delay → phase2attack → phase3hold → phase4decay → phase5sustain
                                                              |
                                                  Note Off    | (or
quickAttack for mode 2)
                                                    ↓         ↓
                                                phase6release ←
phase6quickAttack
                                                    |
                                                    ▼
                                                phase7off (returns true →
voice finished)
```

The state is tracked by `_currentPhase` (enum) and `_currentChunk` (chunk counter within current phase).

Processing Cadence

The envelope is evaluated **once per chunk** (256 samples), producing one value per chunk:

```
// voice.cpp line 161
_volEnvelope.getEnvelope(_dataVolArray, chunkCount, _release,
playedNote,
    FastMaths::fastPow10(0.05f * (_gain - v_attenuation - 0.5f *
```

```
v_filterQ)),
    _voiceParam);
```

This fills `_dataVolArray[0..chunkCount-1]` with one float value per chunk. The same value is used for all 256 samples within that chunk (interpolated linearly — see "volume coefficient" below).

Phase Implementations

Delay (phase1delay)

Output = 0 for `v_timeDelay` chunks:

```
for (quint32 i = 0; i < duration; i++)
    data[chunk++] = 0;
```

Attack (phase2attack) — Volume Envelope

Linear amplitude increase (`lastValue` starts near 0, target is 1):

```
coef = 1.f / v_timeAttack;
for (...) {
    _lastValue += coef;
    data[chunk++] = gain * _lastValue;
}
```

This appears linear, but since the scale is amplitude (not dB), the audible result is a convex (fast start, slow finish) attack — which is more natural-sounding.

Attack (phase2attack) — Modulation Envelope

Asymptotic exponential (convex):

```
float stiffness = 0.000001f;
coef = pow(stiffness, 0.4 / v_timeAttack);
float correction = pow(stiffness, 0.4);
for (...) {
    _lastValue = 1.0f - coef * (1.0f - _lastValue + stiffness +
correction) + stiffness + correction;
    data[chunk++] = gain * _lastValue;
}
```

Hold (phase3hold)

Constant output = gain for `timeHold` chunks.

Key tracking modifies hold time:

```
// enveloppevol.cpp line 95
quint32 timeHold = v_timeHold * FastMaths::fastPow2(v_noteToHold * (60
- note));
```

Where `v_noteToHold` is in units of `cents/key / 1200` (already converted). So at key 60 (middle C), hold = base hold. Higher keys → shorter hold, lower keys → longer hold.

Decay (phase4decay) — Volume Envelope

Exponential decay from full to sustain level:

```
coef = pow(0.00001585 / (1. - levelSustain + 0.00001585), 1. /
timeDecay);
for (...) {
    _lastValue = (_lastValue - levelSustain) * coef + levelSustain;
    data[chunk++] = gain * _lastValue;
}
```

Where `0.00001585 = pow(10, -96/20)`, the minimum audible amplitude (−96 dB). This ensures the decay reaches the sustain level at exactly `timeDecay` chunks.

Decay (phase4decay) — Modulation Envelope

Linear decay:

```
coef = -1.f / timeDecay;
for (...) {
    _lastValue += coef;
    if (_lastValue < levelSustain) _lastValue = levelSustain;
    data[chunk++] = gain * _lastValue;
}
```

Sustain (phase5sustain)

Constant at `levelSustain`:

```
_lastValue = levelSustain;
for (...) data[chunk++] = gain * levelSustain;
```

Sustain continues indefinitely until `release = true`.

Release (phase6release) — Volume Envelope

Exponential decay from current level to silence:

```
coef = pow(0.00001585, 1. / v_timeRelease);
for (...) { _lastValue *= coef; data[chunk++] = gain * _lastValue; }
```

Reaches effectively 0 at `v_timeRelease` chunks.

Quick Release (exclusive class / voice steal)

Forces `v_timeRelease = 0.04 * chunkRate` (40ms):

```
if (_quickRelease)
    v_timeRelease = static_cast<quint32>(0.04f * _chunkRate);
```

Off (phase7off)

Outputs 0. Returns `true` (tells Voice to set `_isFinished = true`).

Volume Coefficient Smoothing

The envelope produces one value per chunk. Within the 256-sample chunk, the volume is **linearly interpolated** to avoid clicks:

```
// voice.cpp lines 286-289
float deltaCoeff = (s_floatConversionCoef24
                    * _dataVolArray[chunk]
                    * FastMaths::fastPow10(0.05f * v_modLfoToVolume *
_modLfoArray[chunk])
                    - _volumeCoeff) * COMPUTATION_CHUNK_INVERT; //
CHUNK_INVERT = 1/256

// Per sample:
_volumeCoeff += deltaCoeff;
(*pData++) *= _volumeCoeff;
```

`COMPUTATION_CHUNK_INVERT = 0.00390625 = 1/256`. This linearly interpolates the volume coefficient over 256 samples from the previous chunk's final value to the current chunk's target value.

Source: `sources/sound_engine/elements/enveloppevol.cpp` (all 314 lines)

Embedded Notes — Envelope Engine

> **ESP32 Relevance:** Very high — the envelope is the most important DSP module.
> The per-chunk evaluation (not per-sample) is essential for ESP32 CPU efficiency.
> Use the same 8-state machine, evaluated once per chunk.
> Volume envelope vs modulation envelope: implement both for a complete sound, but volume envelope is mandatory and modulation envelope is secondary.
> Key tracking for hold/decay (the `60 - note` scaling) is important for authentic musical behavior.
> The `pow()` calls in decay are expensive — pre-compute or approximate for ESP32.

---

10. LFO Engine

Implementation: Triangle Oscillator

Both LFOs (`_modLFO` and `_vibLFO`) are implemented as **triangle wave oscillators**, not sine oscillators. The class is `OscTriangle` .

Why triangle: A triangle wave is computationally trivial (just add a constant and check bounds), yet perceptually close to a sine wave at low modulation frequencies (1–10 Hz). The SF2 spec specifies sine for LFOs, but the triangle approximation is an accepted simplification.

OscTriangle Algorithm

The triangle oscillator maintains:

- `_delta` : increment per sample (positive = going up, negative = going down)
- `_previousPoint` : last output value

```
// osctriangle.cpp lines 77-93 (simplified)
// Going up:
data[total] = _previousPoint + _delta;
if (data[total] > 1.0f) {      // Reached top: bounce
    data[total] = 2.0f - data[total];
    _delta = -_delta;           // Reverse direction
}
_previousPoint = data[total++];
```

The increment per sample:

```
// osctriangle.cpp line 31
_fourInvSampleRate = 4.0f / static_cast<float>(_sampleRate);
_delta = _fourInvSampleRate * freq;  // At sampleRate Hz, completes one
cycle per (1/freq) seconds
```

Why 4? A triangle wave going from 0 → +1 → 0 → -1 → 0 travels a total amplitude of 4 per cycle. So the per-sample increment for one cycle of duration `sampleRate/freq` samples is `4 * freq / sampleRate`.

Frequency Changes

When `freq` changes between chunks, the oscillator smoothly transitions:

```
// osctriangle.cpp lines 65-118
// Gradient interpolation from old delta to new delta across the chunk
progDelta = (remaining * _delta + elapsed * delta2) * invTmp;
data[total] = _previousPoint + progDelta;
```

This avoids discontinuities when frequency is modulated.

Delay

Each LFO has a configurable delay:

```
// osctriangle.cpp lines 44-51
quint32 delayTime = delay * _sampleRate;
if (!_delayEnded && _currentDelay < delayTime) {
    // Output 0 until delay is over
    memset(data, 0, total * sizeof(float));
    _currentDelay += total;
    _delayEnded = (_currentDelay ≥ delayTime);
}
```

During the delay period, LFO output = 0.

Evaluation Cadence

LFOs are evaluated once **per chunk** (256 samples), producing one value per chunk:

```
// voice.cpp lines 169-174
_modLFO.getData(_modLfoArray, chunkCount,
```

```
                _voiceParam->getFloat(champ_freqModLFO),
                _voiceParam->getFloat(champ_delayModLFO));
_vibLFO.getData(_vibLfoArray, chunkCount,
                _voiceParam->getFloat(champ_freqVibLFO),
                _voiceParam->getFloat(champ_delayVibLFO));
```

Output range: [−1, +1]. The chunk value is multiplied by the depth parameter when used.

LFO Routing

The LFO values are applied differently for each destination:

Destination	How Applied	Location
Pitch (vib LFO)	`v_vibLfoToPitch * _vibLfoArray[chunk]` (cents, added to deltaPitch)	voice.cpp L226
Pitch (mod LFO)	`v_modLfoToPitch * _modLfoArray[chunk]` (cents, added to deltaPitch)	voice.cpp L226
Filter cutoff	`v_modLfoToFilterFreq * _modLfoArray[chunk]` (cents, added to log freq)	voice.cpp L294
Volume	`FastMaths::fastPow10(0.05f * v_modLfoToVolume * _modLfoArray[chunk])` (dB → linear)	voice.cpp L288

Source: `sources/sound_engine/elements/osctriangle.cpp`

Embedded Notes — LFO Engine

> **ESP32 Relevance:** Triangle LFO is ideal for ESP32 — nearly zero CPU cost. The per-chunk evaluation (not per-sample) is efficient.
> Key insight: the LFO values are used in three different ways — as cents (pitch), as cents (filter), and as dB gain multiplier (volume). Keep these as separate computations.
> For harmonium simulation, vibLFO and modLFO are musically important — keep both.

---

11. Filter Engine

Filter Type

The filter is a **second-order IIR biquad low-pass filter**, implemented in **Direct Form II**.

This is the standard biquad structure used in audio DSP. It has two feedforward and two feedback coefficients plus a scale coefficient.

Coefficient Computation

Coefficients are computed **once per chunk** using the current filter frequency (which may be modulated):

```
// voice.cpp lines 293-296
float valTmp = v_filterFreq * FastMaths::fastPow2(
    (v_modEnvToFilterFc * _dataModArray[chunk]
     + v_modLfoToFilterFreq * _modLfoArray[chunk]) * 0.000833333f /*
1/1200 */);

float coeffs[3];
if (biQuadCoefficients(coeffs, valTmp, inv_q_lin)) {  ...  apply filter
 ...  }
```

The effective filter frequency = base frequency × 2^(modulation_in_cents / 1200).

Biquad Coefficient Calculation

```
// voice.cpp lines 465-487
bool Voice::biQuadCoefficients(float * coeffs, float freq, float inv_Q)
{
    float theta = 2.f * freq * _audioSmplRateInv;  // Normalized freq
[0, 1] for [0, pi]
    if (theta > 0.98f) theta = 0.98f;              // Safety clamp near
Nyquist

    if (inv_Q ≤ 0.f) return false;                 // Skip filter if Q
invalid

    float dTmp = inv_Q * FastMaths::fastSin(theta) * 0.5f;
    if (dTmp ≤ -1.0f) return false;                // Unstable, skip

    float beta  = 0.5f * (1.f - dTmp) / (1.f + dTmp);
    float gamma = (0.5f + beta) * FastMaths::fastCos(theta);

    coeffs[0] = (0.5f + beta - gamma) * 0.5f;      // b0/a0 = b1/a0 =
b2/a0  (b0=b2=coeffs[0]*2, b1=coeffs[0]*4)
    coeffs[1] = 2.f * gamma;                        // a1/a0
```

```
    coeffs[2] = -2.f * beta;                                    // a2/a0
    return true;
}
```

Note on Q: The input `inv_q_lin` is computed as:

```
float inv_q_lin = FastMaths::fastPow10(-0.05f * (v_filterQ - 3.01f));
// When filterQ = 0: inv_q_lin = pow10(0.15) ≈ 1.41 ≈ sqrt(2)
// This gives a Butterworth response (maximally flat) when Q is 0
// Comment: "If v_filterQ is 0, inv_q_lin is sqrt(2)"
```

Per-Sample Filter Application

```
// voice.cpp lines 298-307
while (pData < pDataEnd) {
    _volumeCoeff += deltaCoeff;                                  //
Advance volume interpolation
    valTmp = coeffs[0] * ((*pData) + _x1 + _x1 + _x2)       // b0*x[n] +
2*b0*x[n-1] + b0*x[n-2]
           + coeffs[1] * _y1                                     //
a1*y[n-1]
           + coeffs[2] * _y2;                                    //
a2*y[n-2]
    _x2 = _x1;                                                  // Delay
line update
    _x1 = (*pData);
    _y2 = _y1;
    _y1 = valTmp;
    (*pData++) = _volumeCoeff * valTmp;                          // Filter
+ volume applied together
}
```

The filter numerator is `coeffs[0] * (x[n] + 2*x[n-1] + x[n-2])`, which represents `b0 = b2 = coeffs[0]` and `b1 = 2*coeffs[0]` (a symmetric filter matching the standard biquad LP formula). Volume multiplication is applied in the same loop.

Filter state variables: `_x1`, `_x2` (input delay), `_y1`, `_y2` (output delay). These persist across buffers (per voice, never reset after initialization).

When the Filter is Bypassed

`biQuadCoefficients()` returns `false` when:

- `theta > 0.98` (above Nyquist) → filter is open/bypassed
- `inv_Q ≤ 0` → degenerate resonance

When bypassed, the simpler volume-only loop runs:

```
// voice.cpp lines 311–323
while (pData < pDataEnd) {
    _volumeCoeff += deltaCoeff;
    (*pData++) *= _volumeCoeff;
}
```

Source: `sources/sound_engine/voice.cpp` lines 465–487, 291–324

Embedded Notes — Filter Engine

> **ESP32 Relevance:** A biquad filter is very practical for ESP32 — only 5 multiplications and 4 additions per sample.
> The per-chunk coefficient update (not per-sample) is a crucial optimization to keep.
> The `fastSin` and `fastCos` calls (for coefficient computation) can be replaced by a lookup table.
> For VHL first version: implement the filter with fixed-coefficient update (once per buffer) for even lower CPU cost.

---

12. Modulator Runtime

Modulator System Overview

At note-on time, modulators are loaded into two `ModulatorGroup` objects per voice:

- `_modulatorGroupInst` — instrument-level modulators (including 10 SF2 default modulators)
- `_modulatorGroupPrst` — preset-level modulators

Each group contains up to `MAX_NUMBER_OF_PARAMETER_MODULATORS = 64` `ParameterModulator` objects.

Modulator Computation at Note-On

When `VoiceParam::computeModulations()` is called, it:

1. Clears all modulation contributions on all 60 parameters (`clearModulations()`)
2. Calls `_modulatorGroupInst.process()` → each `ParameterModulator` computes its output and calls `addInstModulation()` or `addPrstModulation()` on the target `ModulatedParameter`
3. Calls `_modulatorGroupPrst.process()` → same for preset-level

ParameterModulator::computeOutput()

```
// parametermodulator.cpp lines 180-229
bool ParameterModulator::computeOutput()
{
    if (_inputCount < _inputNumber)
        return false;  // Wait for chained inputs

    // Apply curve shapes to inputs
    double input1 = _data.srcOper.applyShape(_input1);
    double input2 = _input2 < 0 ? 0 :
_data.amtSrcOper.applyShape(_input2);

    // Compute modulation amount
    double result = input1 * input2 * _data.amount;

    // Apply absolute_value transform
    if (_data.transOper == SFTransform::absolute_value && result < 0)
        result = -result;

    // Send to output (parameter or chained modulator)
    if (_outputParameter != nullptr) {
        if (_isPrst)
            _outputParameter->addPrstModulation(result);
        else
            _outputParameter->addInstModulation(result);
    }
    _computed = true;
    return true;
}
```

Chained Modulators

When a modulator's `destOper ≥ 32768`, its output feeds another modulator's `input1`. The chain is resolved by repeating the `computeOutput()` loop until all modulators are computed or a maximum iteration count is reached:

```
// modulatorgroup.cpp lines 124-131
bool ok;
int count = 0;
do {
    ok = true;
    for (int i = 0; i < _numberOfParameterModulators; ++i)
        ok &= _modulators[i].computeOutput();
} while (!ok && count++ < _numberOfParameterModulators);
```

Live MIDI Update Path

When MIDI controllers change during playback, the modulator system re-evaluates:

```
// synth.cpp → voiceparam.cpp → modulatorgroup.cpp →
parametermodulator.cpp
bool Synth::processControllerChanged(int channel, int num, int value)
{
    // For all active voices on this channel:
    voiceParam->processControllerChanged(num, value);
}

// VoiceParam::processControllerChanged:
if (_modulatorGroupInst.processControllerChanged(num, value) ||
    _modulatorGroupPrst.processControllerChanged(num, value))
    computeModulations();
```

Each `ParameterModulator` checks if its `srcOper.Index == num` (CC number). If matched, `_input1` is updated and `change = true`. If any modulator changed, `computeModulations()` re-runs the full computation.

ModulatedParameter::computeValue()

When a parameter value is read during audio generation, it computes:

```
// modulatedparameter.cpp lines 115-141
qint32 addition = Utils::round32(_instModulation + _prstModulation)
                + _instValue.getStoredValue().shValue    // instrument
generator value
                + _prstValue.getStoredValue().shValue;   // preset
generator value (additive offset)

// Clamp to int16
if (addition > 32767) _computedValue.shValue = 32767;
```

```
else if (addition < -32768) _computedValue.shValue = -32768;
else _computedValue.shValue = addition;
// Then apply range limits for this attribute type
```

Special case for `initialAttenuation`:

```
// The historical 0.4 coefficient applies to inst and prst stored
values
// but NOT to modulation amounts (which use 0.1 directly)
float value = 0.1f * DB_SF2_TO_REAL_DB * (instStored + prstStored)
            + 0.1f * (_instModulation + _prstModulation);
```

Non-Real-Time Parameters

Some parameters are marked `_notRealTime = true`:

```
champ_keynum, champ_velocity, champ_sampleModes,
champ_scaleTuning, champ_exclusiveClass, champ_overridingRootKey,
champ_keyRange, champ_velRange
```

These are computed once at note-on (the `_computed = true` optimization) and never recomputed, even if modulators change. This makes sense because their values are fixed for the lifetime of a note.

Source: `sources/sound_engine/parametermodulator.cpp`, `sources/sound_engine/modulatedparameter.cpp`

Embedded Notes — Modulator Runtime

> **ESP32 Relevance:** The modulator system is the most complex part of SF2 compliance.
> For VHL first version: hardcode the 10 default modulators as fixed formulas (velocity→attenuation, pitch wheel→pitch, CC7→volume, CC10→pan, etc.).
> Skip custom modulator support entirely in the first version.
> The "live MIDI update → recompute modulators" path is critical for pitch wheel and mod wheel — keep this.
> The distinction between inst modulation and prst modulation (both additive to the stored values) is important.

---

13. DSP Processing Order

Order Within `Voice::generateData()`

```
For each BUFFER (called once per audio callback):
  │
  ▼
Read once: startSamplePos, loopMode, filterQ, filterFreq, keynum

For each CHUNK (256 samples) within the buffer:
  │
  ├─ Step 1: VOLUME ENVELOPE
  │    EnveloppeVol::getEnvelope() → _dataVolArray[chunk]
  │    Evaluates phase state machine, returns one gain value
  │    Also checks if envelope is done → _isFinished = true
  │
  ├─ Step 2: MODULATION ENVELOPE
  │    EnveloppeVol::getEnvelope() → _dataModArray[chunk]
  │    (isMod=true: different shapes, percentage sustain)
  │
  ├─ Step 3: MOD LFO
  │    OscTriangle::getData() → _modLfoArray[chunk]
  │    One value, range [-1, +1]
  │
  ├─ Step 4: VIB LFO
  │    OscTriangle::getData() → _vibLfoArray[chunk]
  │    One value, range [-1, +1]
  │
  ├─ Step 5: PITCH CALCULATION (uses mod env + LFOs)
  │    currentDeltaPitch = deltaPitchFixed
  │                      + modEnvToPitch * modEnvVal
  │                      + modLfoToPitch * modLfoVal
  │                      + vibLfoToPitch * vibLfoVal
  │    fixedGap = round(2^(deltaCents/1200 + 11))
  │
  ├─ Step 6: SINC RESAMPLING
  │    Pull RESAMPLING_ORDER=8 source samples per output sample
  │    Apply 8-tap sinc filter at fractional position
  │    Handle loop boundary in slow path
  │    Output: 256 floats in data[sampleStart..sampleStart+255]
  │    (int16+int24 → float conversion included)
  │
  ├─ Step 7: VOLUME COEFFICIENT (envelope × LFO volume)
  │    deltaCoeff = (s_floatConversionCoef24
  │                  × volEnvVal
  │                  × fastPow10(0.05 × modLfoToVolume × modLfoVal)
  │                  - _volumeCoeff) / 256
  │
  ├─ Step 8: FILTER COEFFICIENT (uses mod env + LFO)
  │    effectiveFreq = baseFreq × 2^((modEnvToFc×modEnv +
```

```
modLfoToFc×modLfo)/1200)
  │    biQuadCoefficients(coeffs[3], effectiveFreq, inv_Q)
  │    → coeffs[0], coeffs[1], coeffs[2]
  │
  └─ Step 9: PER-SAMPLE LOOP (256 samples):
       ├─ _volumeCoeff += deltaCoeff   (volume interpolation)
       ├─ biquad filter (5 mults + 4 adds):
       │    y[n] = coeffs[0]*(x[n] + 2*x[n-1] + x[n-2])
       │         + coeffs[1]*y[n-1] + coeffs[2]*y[n-2]
       │    Update delay lines: x2=x1, x1=x[n], y2=y1, y1=y[n]
       └─ output = _volumeCoeff × y[n]
            (or output = _volumeCoeff × x[n] if filter bypassed)

After all chunks:
  ↓
Voice buffer data[] contains PCM float samples
```

Order Within `SoundEngine::generateData()`

```
For each active Voice:
  │
  ├─ Voice::generateData(voiceBuffer, bufferLen)
  │    (fills voiceBuffer[0..bufferLen-1])
  │
  ├─ Compute pan coefficients:
  │    tmp = 0.005 * (pan + 50)       // Maps [-500,+500] to [0, π/2]
  │    coefL = cos(tmp)
  │    coefR = sin(tmp)
  │
  ├─ Compute reverb/chorus send coefficients:
  │    coefRev    = reverbSend / 100
  │    coefNonRev = 1 - coefRev
  │    coefCho    = chorusSend / 100
  │    coefNonCho = 1 - coefCho
  │
  └─ Accumulate into 8 output buses:
       _dataL      += voiceBuffer × coefL × coefNonRev × coefNonCho
       _dataRevL   += voiceBuffer × coefL × coefRev × coefNonCho
       _dataChoL   += voiceBuffer × coefL × coefNonRev × coefCho
       _dataChoRevL+= voiceBuffer × coefL × coefRev × coefCho
       _dataR      += voiceBuffer × coefR × ...
       (same 4 buses for right channel)
```

Order Within `Synth::readData()`

```
Collect rendered buffers from SoundEngines
    ↓
applyChoRev(): merge 8 buses + apply reverb + chorus effects
    ↓
LiveEQ::filterData() (10-band parametric EQ, editor use only)
    ↓
CalibrationSinus::addData() (pitch calibration sinus, editor use only)
    ↓
FastMaths::clamp() — hard clip both L and R to [-1.0, +1.0]
    ↓
Recorder::process() — save to WAV file if recording
    ↓
Output: dataL[0..maxlen-1], dataR[0..maxlen-1]
```

---

14. Mixer

Voice Summation

All voices are summed into shared float buffers in `SoundEngine`. The first voice uses `multiply()` (copy × coeff), subsequent voices use `multiplyAdd()` (accumulate):

```
// soundengine.cpp lines 182-186
if (leftInitialized)
    FastMaths::multiplyAdd(_dataL, _voiceData, len, coefL * coefNonRev
* coefNonCho);
else {
    leftInitialized = true;
    FastMaths::multiply(_dataL, _voiceData, len, coefL * coefNonRev *
coefNonCho);
}
```

Pan Law

Pan uses **equal-power (cos/sin) panning**:

```
// soundengine.cpp lines 160-162
tmp = 0.005f * (voice->getParam()->getFloat(champ_pan) + 50.f); //
[0.0, 0.5] for [-500, +500]
coefL = FastMaths::fastCos(tmp);  // cos(0..π/2)
coefR = FastMaths::fastSin(tmp);  // sin(0..π/2)
```

- Pan = −500 (full left): `tmp = 0`, `coefL = cos(0) = 1.0`, `coefR = sin(0) = 0.0`
- Pan = 0 (center): `tmp = π/4`, `coefL = coefR = cos(π/4) ≈ 0.707`
- Pan = +500 (full right): `tmp = π/2`, `coefL = 0.0`, `coefR = 1.0`

The equal-power law means the sum `coefL² + coefR² = 1` at all pan positions, giving constant perceived loudness regardless of pan position.

8 Output Buses

Each `SoundEngine` maintains 8 independent float buffers per audio block:

Buffer	Meaning
`_dataL` / `_dataR`	Dry stereo
`_dataRevL` / `_dataRevR`	Reverb-send stereo
`_dataChoL` / `_dataChoR`	Chorus-send stereo
`_dataChoRevL` / `_dataChoRevR`	Both reverb + chorus stereo

Each voice is routed to all four buses simultaneously, scaled by `coefRev` and `coefCho`. This preserves the ability to apply reverb and chorus independently to each voice's contribution.

Multiple SoundEngines

When multiple `SoundEngine` threads exist, the first engine writes its result using `setData()` (memcpy), and remaining engines add their results using `addData()` (vector addition):

```
// synth.cpp lines 705-707
_soundEngines[0]->setData(_dataL, _dataR, ...); // First: copy
for (int i = 1; i < _soundEngineCount; i++)
    _soundEngines[i]->addData(_dataL, _dataR, ...); // Rest: add
```

Clipping

Hard clipping is applied after all mixing:

```
// synth.cpp lines 724-725
FastMaths::clamp(dataL, maxlen);
FastMaths::clamp(dataR, maxlen);
```

`clamp()` clips each sample to [−1.0, +1.0]. This is a hard clipper, not a limiter.

Source: `sources/sound_engine/soundengine.cpp` lines 131–209, `sources/sound_engine/synth.cpp` lines 731–782

Embedded Notes — Mixer

> **ESP32 Relevance:** For ESP32, a single accumulation buffer (mono) is the simplest starting point.
> For stereo: maintain two accumulation buffers (L and R) and apply pan as two multiplications per voice.
> Equal-power pan law: important for musical quality. Can be approximated with a 128-entry cosine lookup table.
> The 8-bus routing for reverb/chorus is not needed on ESP32 — skip entirely.
> Hard clipping: keep. Important for safety.

---

15. Voice Management and Polyphony

Polyphony Limits

```
// From basetypes.h
#define MIN_POLYPHONY 16
#define MAX_POLYPHONY 4096
#define DEFAULT_POLYPHONY 1024
#define MAX_NUMBER_OF_VOICES 2048
```

The active polyphony limit is set by `VoiceList::setMaxPolyphony()` and stored as `_maxPolyphony`.

Voice Management — `VoiceList::prepareComputation()`

This runs once per audio buffer, on the audio callback thread. It is the critical voice management function:

Step 1: Count voices to close

```
int numberOfVoicesToClose = ((additionIndex - first) &
VOICE_INDEX_MASK) - _maxPolyphony;
if (_uncomputedVoiceCount > 0)
    numberOfVoicesToClose += (_maxPolyphony >> 3); // Extra 12.5% when
behind
```

If the total voice count exceeds `_maxPolyphony`, `numberOfVoicesToClose` becomes positive.

Step 2: Gather exclusive class commands

Scans newly added voices (from `lastRunningIndex` to `additionIndex`) and checks `exclusiveClass` (generator 57):

```
// voicelist.cpp lines 160-195
// For each new voice with exclusiveClass ≠ 0:
//   Find any EXISTING voice with same {channel, sf2Id, presetId,
exclusiveClass}
//   If found (and different key): force the old voice to finish
immediately
```

This is how percussion choke groups work: a closed hi-hat forces an open hi-hat to stop.

Step 3: Close finished/released voices

Iterates through all running voices:

```
// voicelist.cpp lines 199-265
close = voice->isFinished()
     || (numberOfVoicesToClose > 0 && voice->isInRelease());

if (close) {
    voice->finish();
    // Swap with first active voice to remove from ring
    _voices[index] = _voices[first];
    _voices[first] = voice;
    first = (first + 1) & VOICE_INDEX_MASK;
    --numberOfVoicesToClose;
}
else if (quickRelease)
    voice->release(true);  // Quick release from exclusive class
```

The swap-and-advance trick removes the voice from the ring without copying arrays — the "dead" voice slot is just pushed to before `_firstRunningIndex`.

Step 4: Force-close oldest voices if still over polyphony limit

If `numberOfVoicesToClose > 0` after closing finished voices, uncomputed voices (those that were added in the current buffer but not yet processed) are forcibly finished, from newest backward. This ensures that polyphony is always enforced.

Voice Stealing

Polyphone's voice stealing strategy (from `prepareComputation()`):

1. First close already-**finished** voices (natural end)
2. Then close voices in **release** phase, oldest first (acceptable audible pop risk is low during release)
3. Then prevent newly-added voices from starting if still over limit (aborted before they produce any audio)

There is no "quiet voice" detection or priority-based stealing. Voices in sustain are never stolen unless the polyphony limit is exceeded by all other means.

Source: `sources/sound_engine/voicelist.cpp` lines 144–275

Embedded Notes — Voice Management

> **ESP32 Relevance:** Voice management is essential for embedded safety. Pre-allocate N voices (e.g., 16) and never exceed this count.
> The exclusive class (choke) mechanism is important for harmonium — a stopped key should choke its associated reed voice.
> Voice stealing strategy: closing releasing voices first is the right choice.
> For ESP32, the ring buffer is overkill — a flat array of N voice structs with `active` flags is simpler and equally correct.

---

16. Runtime Object Reference

`Synth`

File: `sources/sound_engine/synth.h` + `.cpp`

Responsibility: Top-level audio system. Owns all voices, all sound engines, and all effect processors. Bridges the MIDI thread and audio thread. Provides `play()`, `stop()`, and `readData()` entry points.

Key members:

- `VoiceList _voices` — the ring buffer of all pre-allocated voices
- `SoundEngine ** _soundEngines` — array of N worker threads
- `int _soundEngineCount` — `max(CPUthreads - 1, 1)`
- `SynthConfig _configuration` — master gain, reverb/chorus settings, reference pitch, temperament

- `SynthInternalConfig _internalConfiguration` — sample rate, loop/stereo/gain for sample-level playback
- `stk::FreeVerb _reverb` — STK FreeVerb reverb unit
- `stk::Chorus _chorusRevL, _chorusRevR, _chorusL, _chorusR` — 4 chorus instances
- 8 float buffers `_dataL/R/RevL/R/ChoL/R/ChoRevL/R` — per-buffer mix buses

`VoiceList`

File: `sources/sound_engine/voicelist.h` + `.cpp`

Responsibility: Manages the circular ring buffer of all Voice objects. Provides lock-free concurrent access from MIDI thread (preparation) and audio threads (computation). Implements polyphony enforcement and exclusive class handling.

Key members:

- `Voice * _voices[2048]` — all 2048 pre-allocated voice pointers
- `VoiceParam * _voiceParameters[2048]` — parallel array of parameter objects
- `volatile int _preparationIndex, _additionIndex, _firstRunningIndex, _lastRunningIndex`
- `QAtomicInt _currentIndex` — atomic index for multi-thread voice distribution
- `volatile int _maxPolyphony` — adjustable polyphony cap
- `ExclusiveClassCloseCommand _closeCommands[2048]` — pending exclusive class choke commands
- `int _uncomputedVoiceCount` — voices not computed in previous buffer (for load shedding)

`SoundEngine`

File: `sources/sound_engine/soundengine.h` + `.cpp`

Responsibility: Worker thread that generates audio for a subset of voices. Each instance runs on its own `QThread`. Implements the voice rendering loop and accumulation into 8 output buses.

Key members:

- `float * _voiceData` — per-voice temporary buffer (one buffer's worth)

- `float * _dataL/R, _dataRevL/R, _dataChoL/R, _dataChoRevL/R` — 8 accumulation buses
- `QAtomicInt _runningState` — controls start/pause/stop of the computation thread
- `QMutex _mutexSynchro` — gates the start of each computation cycle
- `static VoiceList * s_voices` — shared reference to the voice ring (class-level)

`Voice`

File: `sources/sound_engine/voice.h` + `.cpp`

Responsibility: Complete DSP for one playing note. Contains the sample reader, resampler, volume envelope, modulation envelope, mod LFO, vib LFO, biquad filter, and volume scaling. Self-contained: given a buffer size and pointer, it fills the buffer with its contribution.

Key members:

- `OscTriangle _modLFO, _vibLFO` — LFO oscillators
- `EnveloppeVol _volEnvelope, _modEnvelope` — envelope state machines
- `qint16 * _dataSmpl16` — pointer to 16-bit PCM data
- `quint8 * _dataSmpl24` — pointer to extra-byte PCM data (or = _dataSmpl16 if 16-bit only)
- `quint32 _dataSmplLength` — total sample frames
- `quint32 _smplRate, _audioSmplRate` — source and output sample rates
- `float _gain` — master gain from Synth configuration
- `VoiceParam * _voiceParam` — the parameter object
- `char _loopMode` — current loop mode (0/1/2/3)
- `bool _release, _isFinished` — lifecycle flags
- `quint32 _currentSmplPos` — current integer position in source array
- `quint32 _lastDistanceFraction` — 11-bit fixed-point fractional position (0–2047)
- `qint32 _moreAvailable` — samples remaining until next boundary
- `float _x1, _x2, _y1, _y2` — biquad filter delay lines
- `float _volumeCoeff` — current interpolated volume coefficient
- `float * _dataVolArray, _dataModArray, _modLfoArray, _vibLfoArray` — per-chunk arrays
- `float (*_sincWindow)[RESAMPLING_ORDER]` — selected sinc filter table

- `static float s_sinc_tables[8][256][8]` — all 8 sinc filter tables, shared across voices

`VoiceParam`

File: `sources/sound_engine/voiceparam.h` + `.cpp`

Responsibility: Holds the complete runtime parameter state for one voice. Contains 60 `ModulatedParameter` objects (one per generator), two `ModulatorGroup` objects, and the sample address/loop state. Provides `getFloat()`, `getInteger()`, `getPosition()` accessors used by `Voice::generateData()`.

Key members:

- `ModulatedParameter _parameters[60]` — all 60 generator parameters
- `ModulatorGroup _modulatorGroupInst, _modulatorGroupPrst` — inst and prst modulator processors
- `qint32 _sampleLength, _sampleLoopStart, _sampleLoopEnd, _sampleFineTune` — from `Smpl`
- `qint8 _channel, _key, _type` — voice identification
- `int _sf2Id, _presetId, _wPresetNumber` — voice identification

`ModulatedParameter`

File: `sources/sound_engine/modulatedparameter.h` + `.cpp`

Responsibility: Represents one generator parameter with all its layers: instrument default, preset additive offset, instrument modulation total, and preset modulation total. Computes the final combined value on demand.

Key members:

- `AttributeType _type` — which generator this is
- `Attribute _instValue, _prstValue` — the stored (default + generator) values
- `float _instModulation, _prstModulation` — accumulated modulation amounts
- `bool _notRealTime` — whether to cache the computed value
- `bool _computed, AttributeValue _computedValue` — cached result

`ModulatorGroup`

File: `sources/sound_engine/modulatorgroup.h` + `.cpp`

Responsibility: Manages a group of `ParameterModulator` objects (instrument or preset level). Loads default modulators, handles MIDI event distribution, and runs the modulator computation graph.

Key members:

- `ModulatedParameter * _parameters` — pointer to the shared `_parameters[60]` array in `VoiceParam`
- `bool _isPrst` — whether this is a preset-level group
- `ParameterModulator _modulators[64]` — array of modulator objects
- `int _numberOfParameterModulators` — how many are active

`ParameterModulator`

File: `sources/sound_engine/parametermodulator.h` + `.cpp`

Responsibility: Implements one SF2 modulator. Reads its source (from MIDI state or note-on values), applies its curve shape, multiplies by amount, and writes the result to its target (a `ModulatedParameter` or another `ParameterModulator`).

Key members:

- `ModulatorData _data` — srcOper, amtSrcOper, destOper, amount, transOper, index
- `double _input1, _input2` — current source values
- `ModulatedParameter * _outputParameter` — target parameter
- `ParameterModulator * _outputModulator` — target chained modulator
- `static IMidiValues * s_midiValues` — global MIDI state (shared by all modulators)

`EnveloppeVol`

File: `sources/sound_engine/elements/enveloppevol.h` + `.cpp`

Responsibility: DAHDSR envelope generator. Evaluates one envelope stage per chunk. Returns a per-chunk array of gain values. Handles both volume envelope

(exponential decay/release, linear amplitude attack) and modulation envelope (linear decay/release, convex attack). State machine with 8 states.

Key members:

- `EnveloppePhase _currentPhase` — current state (delay/attack/hold/decay/sustain/quickAttack/release/off)
- `quint32 _currentChunk` — chunk counter within current phase
- `float _lastValue` — last envelope output (used to continue decay/release from current level)
- `quint32 _chunkRate` — chunks per second = `audioSampleRate / 256`
- `bool _isMod` — selects volume vs modulation behavior
- `bool _quickRelease` — forces 40ms release (exclusive class trigger)

`OscTriangle`

File: `sources/sound_engine/elements/osctriangle.h` + `.cpp`

Responsibility: Triangle wave LFO oscillator. Outputs one value per chunk in range [−1, +1]. Supports delay (initial zero period) and smooth frequency changes within a chunk.

Key members:

- `quint32 _sampleRate` — the chunk rate (not audio sample rate)
- `float _fourInvSampleRate` — `4.0 / sampleRate` (constant for increment computation)
- `float _delta` — current per-chunk increment (positive = going up)
- `float _previousPoint` — last output value
- `quint32 _currentDelay` — elapsed chunks during delay phase
- `bool _delayEnded` — delay completion flag

---

17. Source Code Extraction Reference

Entry 1 — Main Note-On Entry Point

Source File	`sources/sound_engine/synth.cpp`
Function	`Synth::play()`

Source File	`sources/sound_engine/synth.cpp`
Lines	130–190
Purpose	Handles a MIDI Note-On event. Routes to preset, instrument, or sample level. Calls `addPreparedVoices()` to atomically publish all new voices.

```
int Synth::play(EltID id, int channel, int key, int velocity)
{
    if (velocity == 0) {          // Note-off encoded as vel=0
        releaseVoices( ... );
        return -1;
    }
    Soundfont * soundfont = _soundfonts->getSoundfont(id.indexSf2);
    switch (id.typeElement) {
    case elementPrst: case elementPrstInst:
        playPrst(soundfont, prst, channel, key, velocity);
        break;
    case elementInst: case elementInstSmpl:
        playInst(soundfont, inst, channel, key, velocity);
        break;
    case elementSmpl:
        playSmpl(smpl, channel, key, velocity, false);
        break;
    }
    _voices.addPreparedVoices(); // Atomic publish
    return playingToken;
}
```

Entry 2 — The Pitch Formula

Source File	`sources/sound_engine/voice.cpp`
Function	`Voice::generateData()`
Lines	201–207
Purpose	Computes the fixed pitch delta in cents, combining sample rate, reference pitch, played key, root key, scale tuning, coarse/fine tune, and temperament.

```
float deltaPitchFixed =
    -1731.234f * qLn(static_cast<double>(_audioSmplRate) * 4400.f /
(_smplRate * s_referencePitch))
  + (playedNote - v_rootkey) * v_scaleTune
  + (temperamentFineTune + static_cast<float>(v_fineTune))
  + 100.0f * v_coarseTune;
```

Entry 3 — Fixed-Point Resampling Ratio

Source File	`sources/sound_engine/voice.cpp`
Function	`Voice::generateData()`
Lines	226–228
Purpose	Converts the current per-chunk pitch delta (in cents, including LFO/envelope modulation) into the fixed-point integer step size for the resampler.

```
float currentDeltaPitch = deltaPitchFixed
    + v_modEnvToPitch * _dataModArray[chunk]
    + v_modLfoToPitch * _modLfoArray[chunk]
    + v_vibLfoToPitch * _vibLfoArray[chunk];

quint32 fixedGap = static_cast<quint32>(
    FastMaths::fastPow2(currentDeltaPitch * 0.000833333f + 11.0f) +
0.5f);
// fixedGap = round(2^(cents/1200 + 11))  where 2^11 = 2048 is the
scale factor
```

Entry 4 — Sinc Resampling Fast Path

Source File	`sources/sound_engine/voice.cpp`
Function	`Voice::generateData()` — label `resample_fast`
Lines	237–262
Purpose	The hot inner loop. Advances the fractional position accumulator, selects the sinc phase, and computes one output sample via 8-tap

Source File	sources/sound_engine/voice.cpp
	dot product.

```
while (true) {
    _lastDistanceFraction += fixedGap;
    if ((qint32)(nextPosition = (_lastDistanceFraction >> 11)) >
_moreAvailable) {
        _lastDistanceFraction -= fixedGap;
        _currentSmplPos += (_lastDistanceFraction >> 11);
        _lastDistanceFraction &= 0x7FF;
        goto resample_slow;
    }
    (*pData++) = FastMaths::multiply8(
        _sincWindow[(_lastDistanceFraction >> 3) & 0xFF],
        &srcData16[nextPosition],
        &srcData24[nextPosition]);
    if (pData ≥ pDataEnd) { /* update position */ goto resample_end; }
}
```

Entry 5 — Loop Wrap Algorithm

Source File	sources/sound_engine/voice.cpp
Function	Voice::getDataWithLoop()
Lines	385–429
Purpose	Advances the sample position while respecting loop boundaries. Wraps around correctly and fills the sinc scratch buffer at loop boundaries.

```
_currentSmplPos += goOn;
while (_currentSmplPos ≥ loopEnd)
    _currentSmplPos += loopStart - loopEnd;   // Wrap

if ((_moreAvailable = loopEnd - _currentSmplPos - RESAMPLING_ORDER) ≥
0) {
    srcData16 = &_dataSmpl16[_currentSmplPos];   // Fast pointer
}
else {
```

```
    // Cross-boundary: copy from end + beginning of loop
    quint32 remainingSamples = loopEnd - _currentSmplPos;
    memcpy(_srcData16, &_dataSmpl16[_currentSmplPos], remainingSamples
* sizeof(qint16));
    memcpy(&_srcData16[remainingSamples], &_dataSmpl16[loopStart],
           (RESAMPLING_ORDER - remainingSamples) * sizeof(qint16));
}
```

Entry 6 — Envelope State Machine

Source File	`sources/sound_engine/elements/enveloppevol.cpp`
Function	`EnveloppeVol::getEnvelope()`
Lines	40–293
Purpose	The complete per-chunk DAHDSR envelope evaluation. Implements all 8 phases with the correct mathematical shape for each.

Key excerpt — exponential decay (volume envelope):

```
// Phase 4: Decay (exponential)
coef = pow(0.00001585 / (1. - levelSustain + 0.00001585), 1. /
timeDecay);
for (...) {
    _lastValue = (_lastValue - levelSustain) * coef + levelSustain;
    data[chunk++] = gain * _lastValue;
}
// 0.00001585 = 10^(-96/20) = amplitude at -96dB
// coef < 1: each chunk _lastValue moves coef-times toward levelSustain
```

Key excerpt — exponential release (volume envelope):

```
// Phase 6: Release (exponential to zero)
coef = pow(0.00001585, 1. / v_timeRelease);
for (...) {
    _lastValue *= coef;
    data[chunk++] = gain * _lastValue;
}
```

Entry 7 — Biquad Filter Coefficients

Source File	`sources/sound_engine/voice.cpp`
Function	`Voice::biQuadCoefficients()`
Lines	465–487
Purpose	Computes the 3 coefficients of a second-order low-pass biquad filter from cutoff frequency and Q. Called once per chunk.

```
float theta = 2.f * freq * _audioSmplRateInv;    // Normalized freq
if (theta > 0.98f) theta = 0.98f;                // Near-Nyquist safety
float dTmp = inv_Q * fastSin(theta) * 0.5f;
float beta  = 0.5f * (1.f - dTmp) / (1.f + dTmp);
float gamma = (0.5f + beta) * fastCos(theta);
coeffs[0] = (0.5f + beta - gamma) * 0.5f;        // b0=b2=coeffs[0],
b1=4*coeffs[0]
coeffs[1] = 2.f * gamma;                          // -a1
coeffs[2] = -2.f * beta;                          // -a2
```

Entry 8 — Per-Sample Filter + Volume Application

Source File	`sources/sound_engine/voice.cpp`
Function	`Voice::generateData()`
Lines	298–324
Purpose	The innermost sample-level DSP loop. Simultaneously applies the biquad LP filter and the linearly interpolated volume envelope.

```
while (pData < pDataEnd) {
    _volumeCoeff += deltaCoeff;                          // Interpolate
volume
    valTmp = coeffs[0] * ((*pData) + _x1 + _x1 + _x2) // Filter
numerator
           + coeffs[1] * _y1 + coeffs[2] * _y2;        // Filter
denominator
    _x2 = _x1;  _x1 = (*pData);                         // Update input
delay
    _y2 = _y1;  _y1 = valTmp;                            // Update
```

```
output delay
    (*pData++) = _volumeCoeff * valTmp;                    // Filter +
volume
}
```

Entry 9 — Mixer: Pan + Bus Routing

Source File	`sources/sound_engine/soundengine.cpp`
Function	`SoundEngine::generateData()`
Lines	159–208
Purpose	Applies equal-power panning and routes each voice into 8 output buses (dry L/R, reverb L/R, chorus L/R, chorus+reverb L/R).

```
tmp = 0.005f * (pan + 50.f);          // Map pan to [0, π/2]
coefL = fastCos(tmp);
coefR = fastSin(tmp);
coefRev = 0.01f * reverbSend;
coefCho = 0.01f * chorusSend;

FastMaths::multiplyAdd(_dataL,      _voiceData, len, coefL * (1-
coefRev) * (1-coefCho));
FastMaths::multiplyAdd(_dataRevL,   _voiceData, len, coefL * coefRev *
(1-coefCho));
FastMaths::multiplyAdd(_dataChoL,   _voiceData, len, coefL * (1-
coefRev) * coefCho);
FastMaths::multiplyAdd(_dataChoRevL,_voiceData, len, coefL * coefRev *
coefCho);
// (same for R channel with coefR)
```

Entry 10 — Exclusive Class (Voice Choke)

Source File	`sources/sound_engine/voicelist.cpp`
Function	`VoiceList::prepareComputation()`
Lines	156–195

Source File	`sources/sound_engine/voicelist.cpp`
Purpose	Detects when a new voice's exclusive class matches an existing voice's class and forces the old voice to finish immediately (voice choke, like hi-hat).

```
// For each newly added voice:
exclusiveClass = voiceParam->getInteger(champ_exclusiveClass);
if (exclusiveClass != 0) {
    // Search existing close commands for a match
    for (int i = 0; i < _closeCommandNumber; i++) {
        if (_closeCommands[i].channel == voiceParam->getChannel() &&
            _closeCommands[i].sf2Id == voiceParam->getSf2Id() &&
            _closeCommands[i].numPreset == voiceParam->getPresetId() &&
            _closeCommands[i].key != voiceParam->getKey() &&      //
Different key
            _closeCommands[i].exclusiveClass == exclusiveClass) {
            // Choke the OLD voice immediately
            _closeCommands[i].voice->finish();
            // Update to point to the new voice
            _closeCommands[i].voice = voice;
        }
    }
}
```

Entry 11 — Polyphony Voice Stealing

Source File	`sources/sound_engine/voicelist.cpp`
Function	`VoiceList::prepareComputation()`
Lines	198–265
Purpose	Enforces the `_maxPolyphony` limit by closing finished voices, then released voices, then newly-added voices.

```
int numberOfVoicesToClose = totalVoices - _maxPolyphony;

for (each running voice, oldest first) {
    close = voice->isFinished()                                    //
Natural end
```

```
            || (numberOfVoicesToClose > 0 && voice->isInRelease());   //
Steal from release
        if (close) {
            voice->finish();
            --numberOfVoicesToClose;
        }
    }
```

Entry 12 — Triangle LFO Core Algorithm

Source File	`sources/sound_engine/elements/osctriangle.cpp`
Function	`OscTriangle::getData()`
Lines	77–157
Purpose	Generates per-chunk triangle wave values with boundary reflection and frequency interpolation.

```
// Ascending phase:
data[total] = _previousPoint + _delta;
if (data[total] > 1.0f) {
    data[total] = 2.0f - data[total];  // Reflect off top
    _delta = -_delta;                   // Reverse
    _previousPoint = data[total++];
    goto down;  // Continue in descending branch
}
_previousPoint = data[total++];
```

Entry 13 — Volume Coefficient Interpolation

Source File	`sources/sound_engine/voice.cpp`
Function	`Voice::generateData()`
Lines	286–289
Purpose	Computes the per-sample delta for smooth linear interpolation of the volume coefficient across a 256-sample chunk, preventing zipper noise.

```
float deltaCoeff = (s_floatConversionCoef24
                    * _dataVolArray[chunk]                              //
Vol envelope value for this chunk
                    * FastMaths::fastPow10(0.05f * v_modLfoToVolume *
_modLfoArray[chunk])  // LFO volume mod
                    - _volumeCoeff)                                     //
Distance from current to target
                   * COMPUTATION_CHUNK_INVERT;                          //
1/256: per-sample step
// Applied per sample: _volumeCoeff += deltaCoeff; sample *=
_volumeCoeff;
```

Entry 14 — ParameterModulator::computeOutput()

Source File	`sources/sound_engine/parametermodulator.cpp`
Function	`ParameterModulator::computeOutput()`
Lines	180–229
Purpose	The core of the SF2 modulator computation. Reads inputs, applies curve shapes, multiplies by amount, and writes to the target parameter or chained modulator.

```
// Apply curve shapes
double input1 = _data.srcOper.applyShape(_input1);
double input2 = _input2 < 0 ? 0 : _data.amtSrcOper.applyShape(_input2);

// Compute modulation
double result = input1 * input2 * _data.amount;
if (_data.transOper == absolute_value && result < 0) result = -result;

// Send to target
if (_outputParameter)
    _isPrst ? _outputParameter->addPrstModulation(result)
            : _outputParameter->addInstModulation(result);
else if (_outputModulator)
    _outputModulator->setInput(result, min, max);
```

---

18. Timing Analysis

When Each Subsystem Runs

Subsystem	Frequency	Trigger
Region selection (preset + inst traversal)	Once per Note-On event	MIDI thread
Voice initialization (`VoiceParam::initialize`)	Once per Note-On per zone	MIDI thread
Modulator loading + first computation	Once per Note-On per voice	MIDI thread
MIDI CC re-computation (`computeModulations`)	Once per CC change	MIDI thread, for all active voices
Envelope evaluation (`getEnvelope`)	Once per chunk (256 samples = 5.8ms at 44100Hz)	Audio worker thread
LFO evaluation (`getData`)	Once per chunk	Audio worker thread
Pitch calculation (`deltaPitchFixed`)	Once per voice per buffer (read from VoiceParam)	Audio worker thread
Per-chunk pitch update (with LFO + env)	Once per chunk per voice	Audio worker thread
Filter coefficient update (`biQuadCoefficients`)	Once per chunk per voice	Audio worker thread
Sinc resampling per output sample	Once per output sample per voice	Audio worker thread
Biquad filter per output sample	Once per output sample per voice	Audio worker thread
Volume coefficient interpolation	Once per output sample per voice	Audio worker thread
Pan computation	Once per buffer per voice	Audio worker thread
Buffer accumulation (mixer)	Once per buffer per voice	Audio worker thread
Reverb + chorus effect processing	Once per buffer	Audio callback thread
Clipping	Once per buffer	Audio callback thread
Voice lifecycle management (`prepareComputation`)	Once per buffer	Audio callback thread

Chunk Rate

At 44100 Hz sample rate with 256 samples per chunk:

```
chunkRate = 44100 / 256 ≈ 172 chunks per second
chunkDuration = 256 / 44100 ≈ 5.8 milliseconds
```

The `COMPUTATION_CHUNK_SHIFT = 8` bit shift is equivalent to division by 256.

The LFO and envelope are parameterized in seconds and converted to chunks at initialization:

```
// enveloppevol.cpp line 64
v_timeDelay = voiceParam→getFloat(champ_delayVolEnv) * _chunkRate;  // seconds → chunks
```

Buffer Size

The audio buffer size is set by the audio server (e.g., 512 or 1024 samples). Polyphone doubles it internally:

```
// synth.cpp line 681
bufferSize *= 2;
```

This gives 2× the audio server's buffer as the internal processing buffer, providing headroom for the double-buffering between audio callback and worker threads.

Embedded Notes — Timing

> **ESP32 Relevance:** The per-chunk cadence is the key design insight.
> On ESP32, a DMA audio buffer of 512 samples at 44100Hz = 11.6ms.
> Process envelopes, LFOs, and filter coefficients once per 256-sample chunk.
> Process filter and volume per sample.
> The chunk boundary can be used to "check MIDI" — no need for MIDI interrupts mid-buffer.

---

19. Runtime Memory

Permanent Allocations (at startup, never freed during playback)

Object	Count	Size
`Voice` objects	2048	~224 bytes each (estimated from members) + sinc buffers
`VoiceParam` objects	2048	60 × `ModulatedParameter` + 2 × `ModulatorGroup`
`ModulatedParameter` per VoiceParam	60 per VP	~40 bytes each
`ParameterModulator` per ModulatorGroup	Up to 64 per group, 2 groups per VP	~80 bytes each
Vol + Mod envelope per Voice	2 per Voice	~32 bytes each
Mod LFO + Vib LFO per Voice	2 per Voice	~24 bytes each
Per-chunk arrays per Voice (`_dataVolArray` etc.)	4 arrays per Voice	`chunkCount × 4 bytes` each
`SoundEngine` internal buffers	8 per SoundEngine	`bufferSize × 4 bytes` each
Sinc tables (shared, static)	1 set total	`8 × 256 × 8 × 4 bytes = 65536 bytes` = 64 KB
Sample data (shared PCM)	1 per loaded Smpl	`frameCount × 2 bytes` (16-bit) or `× 3 bytes` (24-bit)

Per-Voice State (inside each `Voice` object)

State	Size	Lifetime
`_dataSmpl16` pointer	8 bytes (pointer)	Shared with `Smpl::_sound` for type-0 voices
`_currentSmplPos`	4 bytes	Maintained across buffers
`_lastDistanceFraction`	4 bytes	Maintained across buffers
`_moreAvailable`	4 bytes	Updated each chunk
`_x1, _x2, _y1, _y2`	4 × 4 bytes	Filter state, maintained across buffers
`_volumeCoeff`	4 bytes	Current interpolated gain
`_srcData16[8]` , `_srcData24[8]`	8 × 2 + 8 × 1 = 24 bytes	Scratch for boundary resampling

Per-Voice Modulator State

Each active `ParameterModulator` (up to 64 at instrument level, 64 at preset level) holds:

- `ModulatorData _data` — ~20 bytes
- `double _input1, _input2` — 16 bytes
- Pointers to output — 16 bytes
- Total: ~60 bytes per active modulator

With 10 default modulators typically active, that's ~600 bytes per voice for modulators.

Shared, Immutable at Runtime

Data	Shared By	Notes
Sample PCM data (`Sound::_data16`)	All voices using that sample	Read-only during playback
Sinc filter tables (`s_sinc_tables`)	All voices	Static array, computed once at startup
Default modulator data (`s_defaultMods[10]`)	All voices	Static array
`SoundfontManager` data	All voices	Protected by `_mutexSoundfonts` during note-on

Embedded Notes — Runtime Memory

> **ESP32 Relevance:** Memory is the primary constraint.
> 2048 pre-allocated voices is **not** feasible on ESP32. Use 8–32 voices maximum.
> The 64KB sinc table is a problem for ESP32 with 520KB SRAM. Consider using 1–2 sinc tables (linear or simple windowed-sinc).
> Sample PCM data: store in PSRAM (external, 4–8MB on ESP32-S3) or flash. Never in internal SRAM.
> VoiceParam: 60 `ModulatedParameter` objects is too heavy. For VHL, use a simple struct of ~15 floats per voice for the runtime-relevant parameters.
> The filter state (4 floats) is per-voice and tiny — keep it.

---

20. Embedded Notes Summary

This section collects all embedded engineering observations from each section.

The Core Architecture Gap

Polyphone's architecture is designed for:

- Desktop CPUs with 4–16 cores
- Hundreds of MB of RAM
- No real-time hard deadlines (audio glitches are acceptable)
- Maximum quality (8-tap sinc, Qt threading, STK effects)

VHL needs:

- Single ESP32 core (240 MHz)
- ~500KB internal SRAM + optional 4–8MB PSRAM
- Hard real-time: DMA interrupt, no missed deadlines
- Acceptable quality (linear interp, fixed-point, simplified DSP)

Simplifications Table

Feature	Polyphone	VHL First Version
Voice pool	2048 voices, ring buffer	8–32 voices, flat array
Resampling	8-tap sinc, 8 window variants	Linear interpolation (2-tap)
Envelope	Per-chunk, 8 states	Per-chunk, 8 states (keep this)
LFO	Triangle oscillator, per-chunk	Triangle oscillator, per-chunk (keep this)
Filter	Biquad IIR, per-chunk coefficients	Biquad IIR, per-chunk (keep this)
Modulator	Full SF2 modulator graph	Hardcoded default 10 modulators only
Multi-thread	N-1 worker threads	Single ISR or DMA callback
Effects	FreeVerb + STK Chorus	None
Clipping	Hard clamp	Hard clamp (keep this)
Stereo	Full pan law, 8 buses	Two-buffer stereo (L/R), cos/sin pan
Voice stealing	Release-first priority	Release-first priority (keep this)

Feature	Polyphone	VHL First Version
Exclusive class	Full implementation	Full implementation (needed for harmonium)

What Must Be Preserved

1. **Per-chunk DSP cadence** — envelopes, LFOs, filter coefficients once per 256 samples
2. **Per-sample inner loop** — filter + volume per sample
3. **Fixed-point resampling cursor** — integer position + 11-bit fraction
4. **Loop wrap algorithm** — `while (pos ≥ loopEnd) pos += loopStart - loopEnd`
5. **Exponential envelope decay/release** — the geometric decay formula
6. **Equal-power pan law** — cos/sin of normalized angle
7. **Exclusive class choke** — essential for harmonium (key-off choke)
8. **Loop mode 3** — loop during sustain, play to end after release (authentic reed behavior)
9. **Hard clipping** — prevents hardware damage on ESP32 I²S output
10. **Voice stealing from release phase** — prevents polyphony overflow artifacts

---

End of Module 2 — Playback Engine Architecture
Next: Module 3 — VHL Format Design and ESP32 Runtime Implementation